AI ACADEMY · LEVEL 5 · GRADE 10 · WEEK 18 · STUDENT LESSON

Debugging shapes and gradients

Learn to understand, design, build, test and explain AI systems responsibly.

Project: Image-pattern Network Investigation

THE CONNECTED WEEK

Check the shape. Inspect the score. Label the gradient. Verify the update.

Debug structural and semantic errors with complete independent intermediate and coordinate checks.

Keep one habit: Complete dense contract → hidden/exposed bias traces → pictured gradient probes → runnable audit clinic → independent new model.

MAIN LESSON · SECTION 1

Did the shape match—and did every position mean the right thing?

> Check contracts,intermediates,coordinate labels and updates independently.

Learning goals

I can check dimension **contract**s before computing a layer.

I can locate the first wrong value in a forward or gradient trace.

I can verify a repair with independent checks and limited claims.

Retrieve reproducible correctness checks

Close your notes first. A rough first answer helps us choose the right starting point; it is not a score for your ability.

Without opening the old lesson, explain one key idea from Initialisation and reproducibility. Give an example and a mistake that the idea helps you avoid.

Without opening the old lesson, explain one key idea from **Training** and validation curves. Give an example and a mistake that the idea helps you avoid.

Without opening the old lesson, explain one key idea from **Training** a small network. Give an example and a mistake that the idea helps you avoid.

After trying, check the relevant lesson or ask your teacher. Change the part of your explanation that the **evidence** shows was wrong.

Prerequisite and starting response

- Why does running the same code twice and getting the same answer not establish correctness?

> **My starting explanation**

End goal

By the end, I can explain debugging **shape**s and gradients using a traced example and a stated limitation.

▼ Earlier wording, alternatives and picture transcripts

Your learning target

Remember before reading

This week you will practise:

Check dimension **contract**s before computing a layer

Locate the first wrong value in a forward or gradient trace

Verify a repair with independent checks and limited claims

Retrieve what you already know

This week's end goal

MAIN LESSON · SECTION 2

Declare the matrix convention before computing

> One weight-matrix row belongs to one receiving unit in this lesson.

Complete dense layer and batch records

For this lesson, a dense layer stores one **output** unit per weight-matrix row. If an **input** has two features and the layer has three units, W has three rows and two columns, b has length 3, and one **input** x has length 2.

Compute z for each unit, then apply ReLU to each component.

Use W rows [1, 0], [0, 1], [1, -1] and b = [0, 0, 1]. For x = [1, 2], the scores are [1, 2, 0] and the activations are [1, 2, 0].

A batch stores examples in rows: **input**s [[1, 2], [3, 1]] have **shape** 2 by 2, and **output**s have **shape** 2 by 3.

Every shape and its meaning

Object	Declared shape	Meaning
One input x	Length 2	Two features
Weights W	3 by 2	Three units each reading two features
Bias b	Length 3	One bias per unit
Batch output	2 by 3	Two examples each with three activations

Other conventions and semantic axis mistakes

Other software may use a transposed weight convention. The key is to record and follow one **contract**. An array with the right number of values can still be wrong if features or axes are swapped.

Name given and missing facts

> **List the given facts and one fact that is not supplied.**

MAIN LESSON · SECTION 3

Use both hidden and exposed errors to test the repair

> A final activation can agree even when its earlier score is wrong.

Why silent zip truncation needs guards

Python zip stops when its shortest **input** ends. Without a length check, a missing feature can silently remove a contribution.

Reject malformed dimensions before computing: do not pad, truncate or re**shape** a mismatched **input** just to make the calculation run.

Complete correct-versus-zeroed-bias traces

Input; Wrows[1, 0]/[0, 1]/[1, −1] · Correct scores withb[0, 0, 1] · Correct ReLU · Scores ifb 3 set0 · ReLU ifb 3 set0 · What the check reveals

> **Input; Wrows[1, 0]/[0, 1]/[1, −1]:** [1, 2]
>
> **Correct scores withb[0, 0, 1]:** [1, 2, 0]
>
> **Correct ReLU:** [1, 2, 0]
>
> **Scores ifb 3 set0:** [1, 2, −1]
>
> **ReLU ifb 3 set0:** [1, 2, 0]
>
> **What the check reveals:** Score error hidden by activation

> **Input; Wrows[1, 0]/[0, 1]/[1, −1]:** [0, 2]
>
> **Correct scores withb[0, 0, 1]:** [0, 2, −1]
>
> **Correct ReLU:** [0, 2, 0]
>
> **Scores ifb 3 set0:** [0, 2, −2]
>
> **ReLU ifb 3 set0:** [0, 2, 0]
>
> **What the check reveals:** Again same final activation despite wrong score

> **Input; Wrows[1, 0]/[0, 1]/[1, −1]:** [3, 1]
>
> **Correct scores withb[0, 0, 1]:** [3, 1, 3]
>
> **Correct ReLU:** [3, 1, 3]
>
> **Scores ifb 3 set0:** [3, 1, 2]
>
> **ReLU ifb 3 set0:** [3, 1, 2]

> **What the check reveals:** Positive third score exposes the bias bug

Bias-slot,axis and numeric assumptions

In the numeric “omit the third bias”probes, keep its slot and change b=[0, 0, 1]to[0, 0, 0]. That creates a semantic arithmetic bug while all lengths still match. Removing the third slot instead makes bias length 2 and is a different malformed-input case that the guard rejects. The batch contains two row examples and produces a2×3 activation table, not a3×2 table.

Dimension checks do not validate finite numeric values, feature order or intended units.

The function assumes finite numeric values; it checks dimensions, not every possible **input** fault. Evaluate the two batch rows independently and stack their **output**s.

Retain tests that reject a missing **input** feature, a short weight row and an incorrect bias count.

Complete runnable guarded dense layer

```
def dense_relu(x, weights, bias):
    if not x or not weights:
        raise ValueError("Need input features and units")
    if len(bias) != len(weights):
        raise ValueError("One bias per unit required")
    if any(len(row) != len(x) for row in weights):
        raise ValueError("Each unit needs one weight per feature")
    scores = [sum(w * value for w, value in zip(row, x)) + b
              for row, b in zip(weights, bias)]
    return scores, [max(0.0, z) for z in scores]

weights = [[1, 0], [0, 1], [1, -1]]
bias = [0, 0, 1]
for x in [[1, 2], [3, 1]]:
    print(dense_relu(x, weights, bias))
# ([1, 2, 0], [1, 2, 0])
# ([3, 1, 3], [3, 1, 3])
```

Connect the shape diagram to named arrays

Point to the **input**, the deciding step and the result in this week's visual. Explain the same step in ordinary words. A picture helps when you can say what each part represents.

▼ Earlier wording, alternatives and picture transcripts

Connect the picture and the explanation

MAIN LESSON · SECTION 4

Keep shape checks separate from numerical and semantic checks

> Passing one kind of test does not establish the other kinds.

Current exposed-bias task

- Compute scores and activations for x = [3, 1] with the lesson layer, then omit the third bias. Identify the first discrepancy and the resulting activation error.

> **Keep the procedure fixed. Explain what changed in the inputs and result.**

Predict the pictured declared-order change

- Change: The program changes its declared order to [c, a] and returns [-12,-8].

▼ **Check after predicting**

Prediction: That vector now matches the declared coordinate order.

Reason: Gradient values are meaningful only when their parameter names and array positions agree.

▼ **Earlier wording, alternatives and picture transcripts**

Compare a familiar case with a changed case

Change one thing in the pictured case

MAIN LESSON · SECTION 6

Label each gradient entry before an optimizer uses it

> Two entries can have the correct length and the wrong coordinate meaning.

Complete pictured scalar facts and separate scope

Catch a gradient -order bug before the optimiser changes parameters.

Toy y=2a+3c, target 4, start [a, c]=[1, 0]. Loss $(y-4)^2$. Program claims gradient [-12,-8] in order [a, c].

All **input**s and outcomes in this worked example are invented classroom data; no real model run is claimed.

The picture is a separate scalar gradient model:y=2a+3c, target 4, old[a, c]=[1, 0], y2, residual−2. Correct ordered gradient[−8, −12]is mislabeled by a program returning[−12, −8]under[a, c]. With spacing.1, a-minus/plus losses 4.84/3.24 give−8; c-minus/plus 5.29/2.89 give−12. Rate.01 with the correct old-state gradient gives[1.08, .12], y2.52, loss 2.1904.

Under a genuinely changed order[c, a], the same gradient is correctly represented[−12, −8], provided every consumer follows that order.

Every minus/plus coordinate probe

Pictured coordinate probe · Held coordinate · Minus vector /loss · Plus vector /loss · Difference over.2

> **Pictured coordinate probe:** a
>
> **Held coordinate:** c0
>
> **Minus vector /loss:** [.9, 0]/4.84
>
> **Plus vector /loss:** [1.1, 0]/3.24
>
> **Difference over.2:** -8

> **Pictured coordinate probe:** c
>
> **Held coordinate:** a1
>
> **Minus vector /loss:** [1, −.1]/5.29
>
> **Plus vector /loss:** [1, .1]/2.89
>
> **Difference over.2:** -12

Trace the correct gradient and update

AI ACADEMY / AI-5/18 · 02 · TRACE THE CASE

Follow the deciding steps

01 Compute correct gradient

y = 2, residual -2: gradient [-8,-12], not [-12,-8].

02 Check coordinates

Step 0.1: a losses 3.24, 4.84 give -8; c losses 2.89, 5.29 give -12.

03 Apply correct update

At rate 0.01: new [a, c]=[1.08, 0.12]; y = 2.52, loss 2.1904.

Follow the labelled picture with the matching explanation. Open the vector diagram

Read the picture one step at a time

STEP	WHAT HAPPENS AND WHY
1. Compute correct gradient	y = 2, residual -2: gradient [-8,-12], not [-12,-8].
2. Check coordinates	Step 0.1: a losses 3.24, 4.84 give -8; c losses 2.89, 5.29 give -12.
3. Apply correct update	At rate 0.01: new [a, c]=[1.08, 0.12]; y = 2.52, loss 2.1904.

Match each pictured step to the supplied facts. This is a worked teaching case, not an independent assessment.

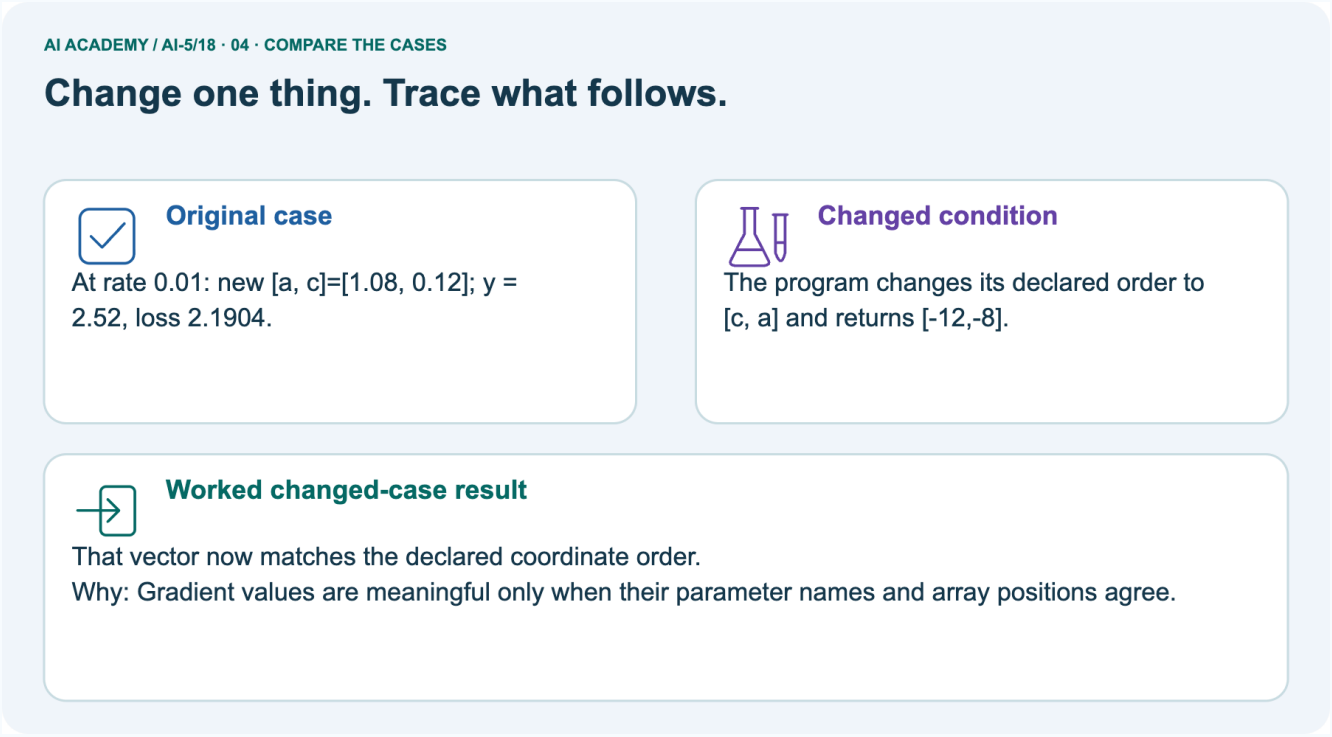
Array **shape** and coordinate meaning both matter. A vector with two entries can have the right length but assign each derivative to the wrong parameter.

Label the entries a and c, then perturb one coordinate at a time to check their slopes. Repairing the coordinate mapping is necessary before using the gradient to update the network.

Predict the changed declared order

- Change one thing: The program changes its declared order to [c, a] and returns [-12,-8]. Predict the result before checking the explanation. What remains the same?

▼ Check after predicting



Follow the labelled picture with the matching explanation. Open the vector diagram

Explain coordinate labels and check limits

- A correct gradient at one point does not establish every batch, **shape** or activation path is implemented correctly.

- Read the labelled picture: What should a debugging record pair with every gradient value? Point to the relevant step in the picture, then write the changed result and the rule or **evidence** that supports it.

Return to the main layer's hidden-bias comparison

Repair from the first disagreement

TRACE STAGE FOR X [1 2]	EXPECTED RESULT	SUBMITTED RESULT
Input and dimensions	x length 2 W 3 by 2 b length 3	Dimensions match
Scores before ReLU	[1, 2, 0]	[1, 2, -1]
Hidden activations	[1, 2, 0]	[1, 2, 0]

The submitted third score omits bias 1. ReLU hides the error for this input, so matching final activations are not enough. At x = [3, 1], the same omission changes the third activation from 3 to 2.

Trace x = [0, 2] with the correct layer. Then omit the third bias. Which scores differ, and why do the activation vectors still agree?

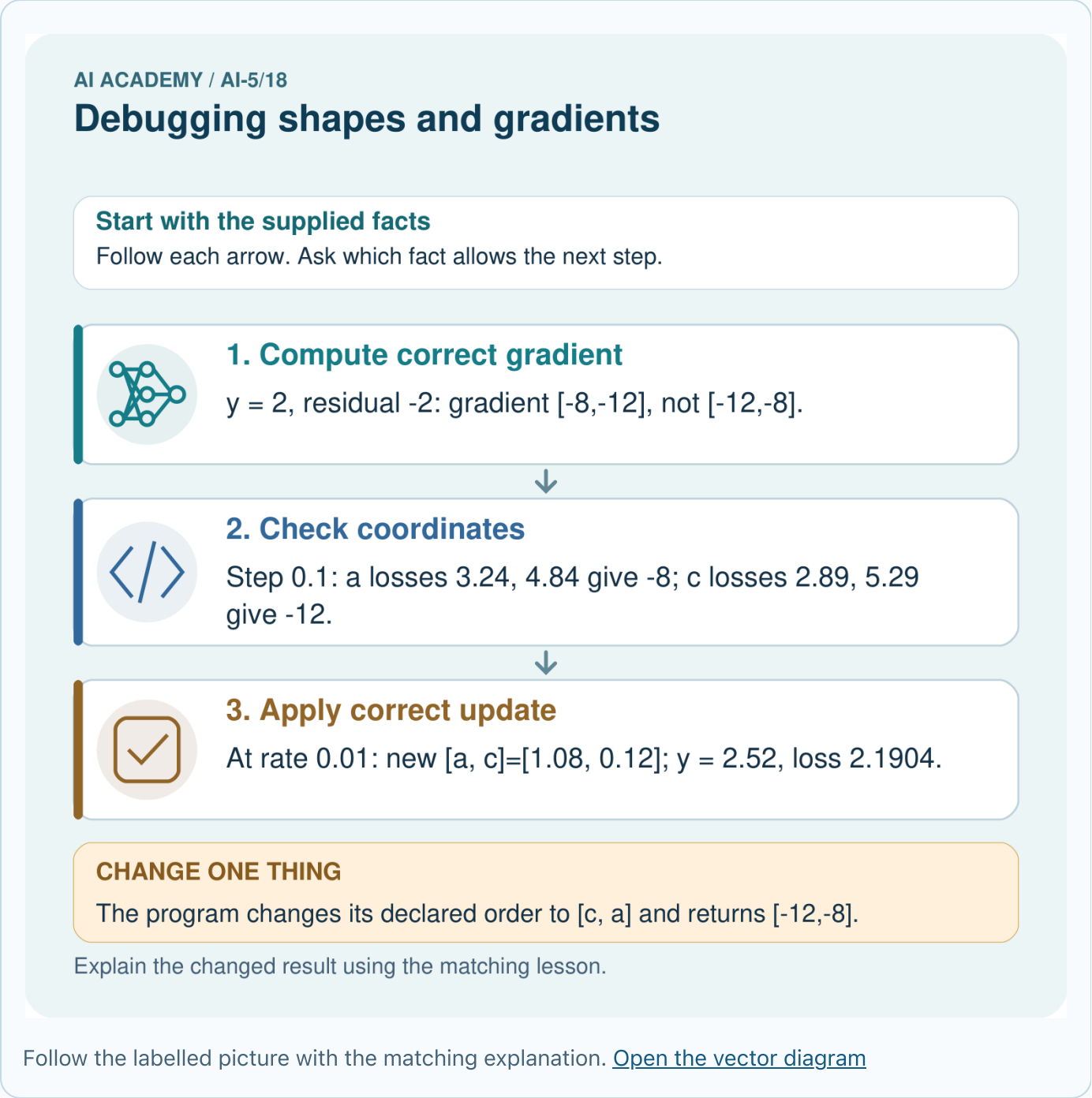
▼ **Earlier wording, alternatives and picture transcripts**

Learn from the worked case

Cover the result before checking it. Say what is given, choose the procedure, and follow one step at a time. Then uncover the result and explain your first mismatch. Ask why a step is allowed, not only what number it produces.

A pictorial worked case: Debugging shapes and gradients

The facts we will use



Follow the labelled picture with the matching explanation. Open the vector diagram

Why this result follows

What this example does not establish

MAIN LESSON · SECTION 7

Repair the mapping before trusting a loss decrease

> The scalar diagnostic has its own formula and coordinate order.

Complete separate diagnostic and numerical assumptions

Use a separate scalar diagnostic model y = 2w + b, target 3, loss (y - 3) squared, and parameter order [w, b]. At [1, 0], **prediction** is 2, residual -1 and gradient [-4, -2].

A submitted gradient [-2, -4] has the correct length but swapped meanings.

With symmetric step 0.1, probe w while b stays 0: losses at w = 0.9 and 1.1 are 1.44 and 0.64. Their difference divided by 0.2 is -4. Probe b while w stays 1: losses at b = -0.1 and 0.1 are 1.21 and 0.81, giving -2.

Use original parameters for both probes and compare by coordinate with a stated tolerance.

A gradient check must include the actual loss convention and any penalty. For ReLU, avoid probes crossing its corner at zero; a symmetric value there need not be a derivative.

A passing numerical check at selected points is **evidence** about those points, not a proof for every **input**.

Compare correct subtraction with a sign bug

- For the diagnostic model at [1, 0], gradient [-4, -2] and rate 0.05, calculate the correct new parameters and loss. Compare with adding the gradient step.

> **State one repair and the checks you would rerun.**

▼ Earlier wording, alternatives and picture transcripts

Locate the earliest unsupported step

MAIN LESSON · SECTION 8

Practise first-error detection with less help

> Retain the expected intermediate value and the test that exposes it.

Current task and response

- The layer expects W **shape** 3 by 2, b length 3 and x length 2. A submitted x has length 3. Should the last feature be silently ignored? State the failed **contract** and repair **process**.

Trace x = [0, 2] with the correct layer. Then omit the third bias. Which scores differ, and why do the activation vectors still agree?

Explain your prediction to a partner before comparing results.

Supported W2record

Use workbook W2 for the connected example “Debugging **shape**s and gradients”. First fill the **input** and rule boxes. Try the middle steps before asking for a hint. After a hint, try the next step yourself.

Plan	Do	Check
What is given? Which rule or procedure applies?	Record each intermediate step.	Does the result follow? What remains unknown?

Explain your trace to a partner. Your partner points to one step to check. Swap roles on the next case. Each person writes their own explanation.

Individual misconception check

- Use the worked case for Debugging **shape**s and gradients. Proposed explanation: “Matching **output shape** guarantees the computation”. Decide whether this explanation is supported.

  Point to the step or supplied fact that decides; explain your repair if needed.

- Answer on your own before discussion. If you are unsure, say which step you need help with.

Pictured gradient-label reasoning

- What should a debugging record pair with every gradient value? Point to the relevant step in the picture, then write the changed result and the rule or **evidence** that supports it.

> **My explanation using a pictured step and the given facts**

Reserve the independent new gradient model

This is supported practice. Keep the separately named independent case for an attempt before feedback.

▼ Earlier wording, alternatives and picture transcripts

Practise with less help: Expose a hidden bias error

Check your reasoning

Explain the picture

MAIN LESSON · SECTION 9

Run a complete shape,hidden-error and gradient audit clinic

> All arrays,submitted errors,coordinate probes and repair expectations are supplied.

Current main-layer task and guarded function

Compute scores and activations for x = [3, 1] with the lesson layer, then omit the third bias. Identify the first discrepancy and the resulting activation error.

```
def dense_relu(x, weights, bias):
    if not x or not weights:
        raise ValueError("Need input features and units")
    if len(bias) != len(weights):
        raise ValueError("One bias per unit required")
    if any(len(row) != len(x) for row in weights):
        raise ValueError("Each unit needs one weight per feature")
    scores = [sum(w * value for w, value in zip(row, x)) + b
              for row, b in zip(weights, bias)]
    return scores, [max(0.0, z) for z in scores]

weights = [[1, 0], [0, 1], [1, -1]]
bias = [0, 0, 1]
for x in [[1, 2], [3, 1]]:
    print(dense_relu(x, weights, bias))
# ([1, 2, 0], [1, 2, 0])
# ([3, 1, 3], [3, 1, 3])
```

Complete separate gradient-coordinate program

```
# Separate pictured diagnostic: y = 2a + 3c, target 4; not the dense layer.
def loss(a, c):
    return (2*a + 3*c - 4)**2

def partials(a, c, h=.1):
    if h <= 0:
        raise ValueError('Need positive finite-difference spacing')
    return [(loss(a+h,c)-loss(a-h,c))/(2*h),
            (loss(a,c+h)-loss(a,c-h))/(2*h)]

old = [1.,0.]
labels = ['a','c']
reported = [-12.,-8.]  # Correct length, wrong coordinate assignment.
checked = partials(*old)
formula = [-8.,-12.]
tolerance = 1e-6
for name, bad, numerical, correct in zip(labels,reported,checked,formula):
    print(name, 'reported/numerical/formula', bad, round(numerical,6), correct)
    print('reported entry agrees', abs(bad-numerical)<tolerance)
assert all(abs(n-f)<tolerance for n,f in zip(checked,formula))
new = [p-.01*g for p,g in zip(old,formula)]
print('correct new [a,c]', [round(p,6) for p in new])
print('prediction/loss', round(2*new[0]+3*new[1],6), round(loss(*new),6))
print('same gradient in [c,a] order', [formula[1],formula[0]])
# Finite numeric values assumed; these probes do not cross an activation corner.
```

Normal,boundary,malformed,missing and sign probes

Complete debugging probe	Expected result / action
Normal two-row batch	Outputs[[1, 2, 0], [3, 1, 3]], shape 2×3
Boundary mainx[1, 2], third score 0	Zeroingb3 gives–1 but ReLUstill0; check scores, not only activations
Input feature missing:x[1]with originalW	ValueError:Each unit needs one weight per feature
Short third weight row[1]	Same dimension ValueError

Complete debugging probe	Expected result / action
Bias length 2 for3 units	ValueError:One bias per unit required
Empty input or no weight rows	ValueError:Need input features and units
Missing numeric value in valid-length input	Outside finite-number contract; review before calling
Pictured gradient[−12, −8]under[a, c]	Both entries disagree with labelled numerical probes; repair to[−8, −12]
Diagnostic[w, b]model, rate.05	Subtract verified[−4, −2]→[1.2, .1], loss.25; adding gives[.8, −.1], loss 2.25
One repaired output or one decreased loss	Still retain shape guards, independent intermediate traces, coordinate checks and saved-state update checks

Predict,run or trace,repair and retain checks

1. Predict the complete correct/zeroed-bias traces, then run or trace dense_relu on each valid and malformed case separately. Preserve all errors and score vectors. Next use the complete separate gradient program to audit coordinate labels and recompute the new loss. Do not pad/truncate arrays to silence an error or accept a decreasing loss as proof of the requested algorithm.

   The function assumes finite numerical values; screen missing values first. Keep the independent different model for a first attempt before hints.

2. On paper, trace the supplied procedure. If code is provided, predict before running it in a local practice file. Never write an expected answer as though it were an observed run.

Keep the evidence ledger

Record	Your evidence
Given input and fixed rule	

Record	Your evidence
Prediction before checking	
Actual result, or “not run”	
First mismatch and one explanation	
Boundary or missing input and expected response	

Before trying: what do I expect, and why? While trying: which step could first go wrong? Afterwards: what did I actually observe, and what should change? Keep “not run” if you only traced the procedure.

▼ **Earlier wording, alternatives and picture transcripts**

Predict, investigate and preserve the evidence

Plan → try → check

MAIN LESSON · SECTION 14

Try alone: check and repair a different ordered gradient

> Use the new model’s sensitivities and its own stated learning rate.

Complete new independent gradient and update task

Use y = 3a + c, target 2, loss (y - 2) squared, at [a, c] = [0, 1]. A program returns gradient [-2, -6] in order [a, c].

Use symmetric step 0.1 to check each coordinate, then update both with the corrected gradient and rate 0.05. Report new parameters and loss.

First attempt and support record

> **My independent explanation, trace and limits**

Attempt this case before feedback. Record any help. Earlier examples below are further practice; a case already practised with feedback cannot establish fresh transfer.

Use feedback after trying

Attempt the independent case before hints. Record whether you worked alone, used a hint or followed a model. After feedback, fix the first unsupported step and explain why it changed.

A later check uses a changed case, not a copied answer.

▼ **Earlier wording, alternatives and picture transcripts**

Independent case: Debug a new gradient independently

Attempt this without hints

Use y = 3a + c, target 2, loss (y - 2) squared, at [a, c] = [0, 1]. A program returns gradient [-2, -6] in order [a, c].

Use symmetric step 0.1 to check each coordinate, then update both with the corrected gradient and rate 0.05. Report new parameters and loss.

> **My independent procedure, result and limitation**

Record support used: none, hint or teacher explanation. After feedback, a different case is needed to demonstrate independence.

MAIN LESSON · SECTION 11

Keep the failed contract and verified repair in the project record

> Another reader should see the first wrong value and the checks used after repair.

Project artifact and evidence table

Add a labelled debugging **shape**s and gradients **evidence** sheet to Image-pattern Network Investigation. Show how you can check dimension **contract**s before computing a layer.

Use the supplied fictional case and keep its **input**s, intermediate steps, calculation or **prediction**, and limitation. Connect this record to last week's record; a matching answer without a trace is insufficient.

Project evidence	Record
Component and version	
Inputs permitted at prediction time	
Trace and comparison with earlier work	
Failure, limit and human review	

Actual run/trace and remaining limits

For your Image-pattern Network Investigation, save the **input**, procedure, observed result and limitation of this check.

Label this worked example as practice; use a different, previously unreviewed case when checking independent understanding.

▼ Earlier wording, alternatives and picture transcripts

Evidence for your project

MAIN LESSON · SECTION 16

Finish with one revealing test and one bounded correctness claim

> Explain which type of error each retained check can expose.

Retrieve and explain

- Close your notes. Explain this goal in your own words: Check dimension **contract**s before computing a layer

> **My deciding step, evidence and one remaining question**
>
> | | |
> |---|---|
> | | |
> | | |
> | | |
> | | |

Plan a changed-case return

If you are unsure, return to the worked picture and explain one arrow. If you are ready, change an **input** or assumption and justify your **prediction**.

Save your workbook **evidence**. At the next review, try a different case before opening your old answer.

Reference bank and extra practice

Choose the reference that helps your next step.

▼ **Reference · Use the mathematical words precisely**

REFERENCE · SECTION 5

Reference · Use the mathematical words precisely

> Connect each term to a value or step in the current example.

Vocabulary and your explanation

Word	Meaning
Shape	The sizes of an array along its declared axes
Contract	A stated condition that inputs and outputs must satisfy
Partial gradient	The change rate of a scalar loss with respect to one parameter while others are fixed
Regression check	A saved check rerun after a repair to detect the return of a defect

Use two terms to explain a step in this week's example.

▼ Extra challenge · Explain a stronger case

REFERENCE · SECTION 10

Extra challenge · Explain a stronger case

> Choose the next challenge after the main route.

Explain,repair and extend

- The layer expects W **shape** 3 by 2, b length 3 and x length 2. A submitted x has length 3. Should the last feature be silently ignored? State the failed **contract** and repair **process**.
- For the diagnostic model at [1, 0], gradient [-4, -2] and rate 0.05, calculate the correct new parameters and loss. Compare with adding the gradient step.
- A program passes only the x = [1, 2] final-activation test. Propose one intermediate-value check, one malformed-**input** check and one additional **input** that exposes the bias omission.

Explain → apply → challenge

> **Explain which evidence supports your conclusion and which claim remains unproved.**

▼ Reference · Check the representation and its limits

REFERENCE · SECTION 12

Reference · Check the representation and its limits

> Use after practice;reserve the new independent y3a+c,target2gradient repair for the first individual attempt.

Current explanation and response

- At [w, b] = [1, 0], use the verified gradient [-4, -2] and rate 0.05. Subtracting rate times gradient gives [1.2, 0.1], **prediction** 2.5 and loss 0.25. A sign bug that adds it gives [0.8, -0.1], **prediction** 1.5 and loss 2.25.

  Recompute loss after the complete update instead of trusting an intended direction.

- Lower loss alone cannot prove a correct gradient or update; some wrong steps also improve loss. Retain **shape** rejections, hand-computed intermediate traces, numerical coordinate checks and update checks.

  Next week investigates the representations produced by hidden layers.

- A program passes only the x = [1, 2] final-activation test. Propose one intermediate-value check, one malformed-**input** check and one additional **input** that exposes the bias omission.

> **A repair makes one final output match but leaves the third score wrong. Another repair makes loss decrease but has swapped gradient entries. Are either of these sufficient implementation checks? Describe the evidence needed.**

My limited claim and a useful next check

▼ **Extra practice · Revisit the complete trace**

REFERENCE · SECTION 13

Extra practice · Revisit the complete trace

> Predict before reading the supplied steps.

Complete trace and evidence

Complete evidence record

Repair from the first disagreement

TRACE STAGE FOR X [1 2]	EXPECTED RESULT	SUBMITTED RESULT
Input and dimensions	x length 2 W 3 by 2 b length 3	Dimensions match
Scores before ReLU	[1, 2, 0]	[1, 2, -1]
Hidden activations	[1, 2, 0]	[1, 2, 0]

The submitted third score omits bias 1. ReLU hides the error for this input, so matching final activations are not enough. At x = [3, 1], the same omission changes the third activation from 3 to 2.

Follow the **input** through the stated procedure to its result. Cover the result first, predict it, then explain every intermediate step. The text/table accompanies the figure so the example can be read without colour.

My complete trace of the worked case

▼ Reference · The earlier route checklist

REFERENCE · SECTION 15

Reference · The earlier route checklist

> Use one main route rather than repeating the lesson.

Targets,route and first prediction

Start with this question: Catch a gradient -order bug before the optimiser changes parameters.

By the end, I can	How I will show it
Check dimension contracts before computing a layer	Explain the deciding step, show a trace or test, and state one limit.
Locate the first wrong value in a forward or gradient trace	Explain the deciding step, show a trace or test, and state one limit.
Verify a repair with independent checks and limited claims	Explain the deciding step, show a trace or test, and state one limit.

Remember → watch and explain → try with help → investigate → try independently → keep project **evidence**. Your teacher may spread this lesson across several classes.

Keep your first **prediction**. Compare it with what the **evidence** shows and explain any change.

▼ Extra practice · Predict,check and repair

REFERENCE · SECTION 17

Extra practice · Predict,check and repair

> Keep supported feedback separate from the independent case.

First supported question and optional hint

- Catch a gradient -order bug before the optimiser changes parameters.
- Which labelled step matches this detail? At rate 0.01: new [a, c]=[1.08, 0.12]; y = 2.52, loss 2.1904.
- 1. Compute correct gradient
- 2. Check coordinates
- 3. Apply correct update
- Hint if needed: Read the steps in the pictorial case. Match the supplied detail to the stage that performs or records it.

Practice 1: explain the deciding evidence

My selected conclusion, deciding fact and explanation

▼ **Check after the first attempt**

After your attempt, compare with the supported explanation: Apply correct update At rate 0.01: new [a, c]=[1.08, 0.12]; y = 2.52, loss 2.1904. Array **shape** and coordinate meaning both matter.

A vector with two entries can have the right length but assign each derivative to the wrong parameter. Label the entries a and c, then perturb one coordinate at a time to check their slopes.

Repairing the coordinate mapping is necessary before using the gradient to update the network.

Changed-case question and optional hint

- Changed-case practice: The program changes its declared order to [c, a] and returns [-12,-8]. Which conclusion is supported by the supplied facts?
- 1. This one example guarantees correct results for every future **input**.
- 2. That vector now matches the declared coordinate order.
- 3. We can decide the result without examining the changed **input** or the procedure.
- Hint if needed: Use the changed condition and the deciding step in the pictorial trace. The answer must say what follows from those facts.

Practice 2: explain the deciding evidence

> **My selected conclusion, deciding fact and explanation**
>
> | |
> |---|
> | |
> | |
> | |

▼ **Check after the changed attempt**

After your attempt, compare with the supported explanation: That vector now matches the declared coordinate order. Gradient values are meaningful only when their parameter names and array positions agree.

Repair and retain the record

Review Initialisation and reproducibility (ai-5/17). Goal: Trace how starting values affect a fixed **training** budget. Short practice: Does the matching final loss mean the parameter values are equal?

Point to the relevant step in the picture, then write the changed result and the rule or **evidence** that supports it.. Return to this week and explain what you changed.

Review **Training** and validation curves (ai-5/15). Goal: Compute comparable **training** and validation losses at saved checkpoints. Short practice: Can the original epoch 3 **training** loss 1 justify claiming best generalisation?

Point to the relevant step in the picture, then write the changed result and the rule or **evidence** that supports it.. Return to this week and explain what you changed.

Review **Training** a small network (ai-5/12). Goal: Trace repeated forward loss gradient and update steps. Short practice: Which parameter changed, and which part was not trained?

Point to the relevant step in the picture, then write the changed result and the rule or **evidence** that supports it.. Return to this week and explain what you changed.

The browser lab lets you edit the supported Python examples already included in this lesson. Predict first, run, inspect the actual **output** or error, change one **input**, then run again.

Keep code, **prediction**s, actual results and a reasoned revision together. On paper, label your result as a trace rather than an execution.

Use Download completed work in the hosted lesson to export your answers, reflections, practice feedback and code-run **evidence**. Open the HTML copy to read or print it.

Drafts stay in this browser when storage is available; clear drafts on a shared device.

▼ Earlier wording, alternatives and picture transcripts

Use this supported practice once, in the browser or on paper. Keep the reserved independent assessment for a separate first attempt before teacher feedback.

Repair a missing building block

Predict, run and debug

Keep your completed work

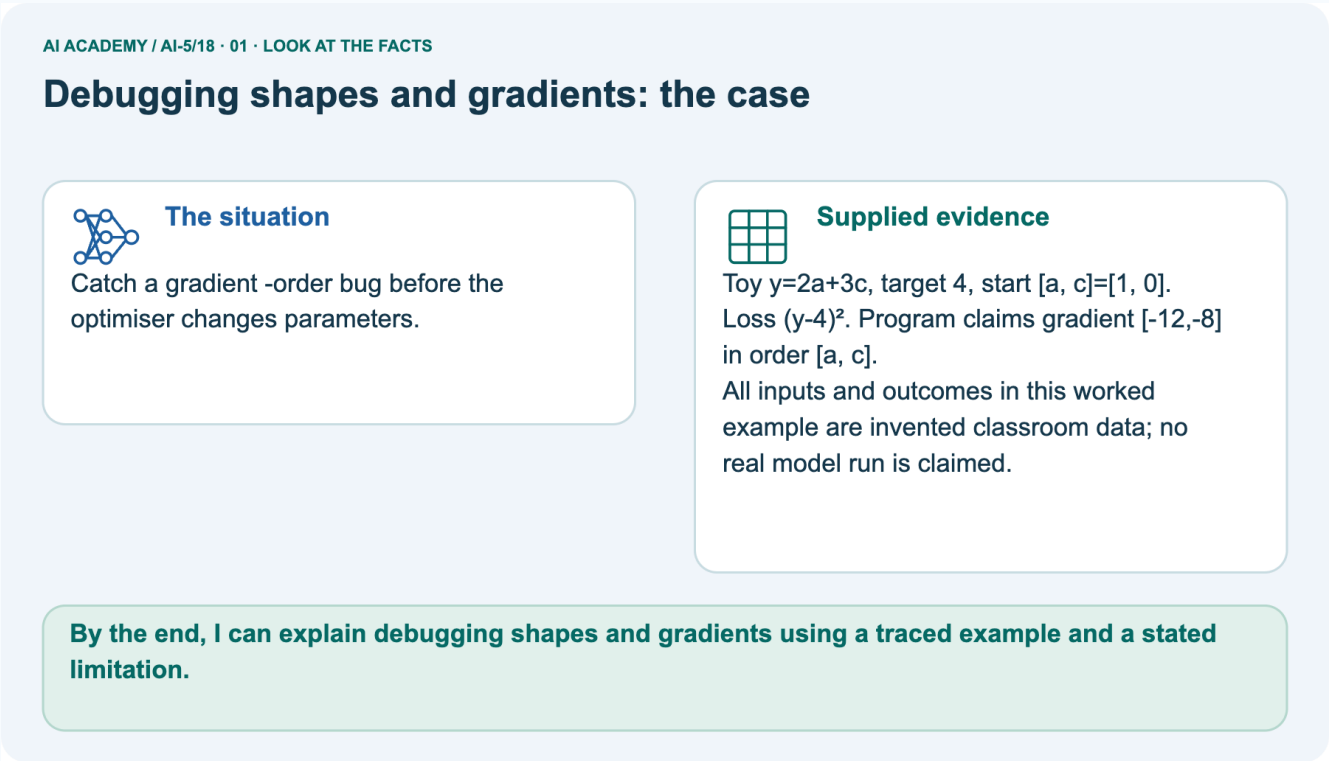
▼ Reference · Six-picture concept journey

REFERENCE · SECTION 18

Reference · Six-picture concept journey

> Choose the picture that supports the current explanation.

01. Debugging shapes and gradients: the case



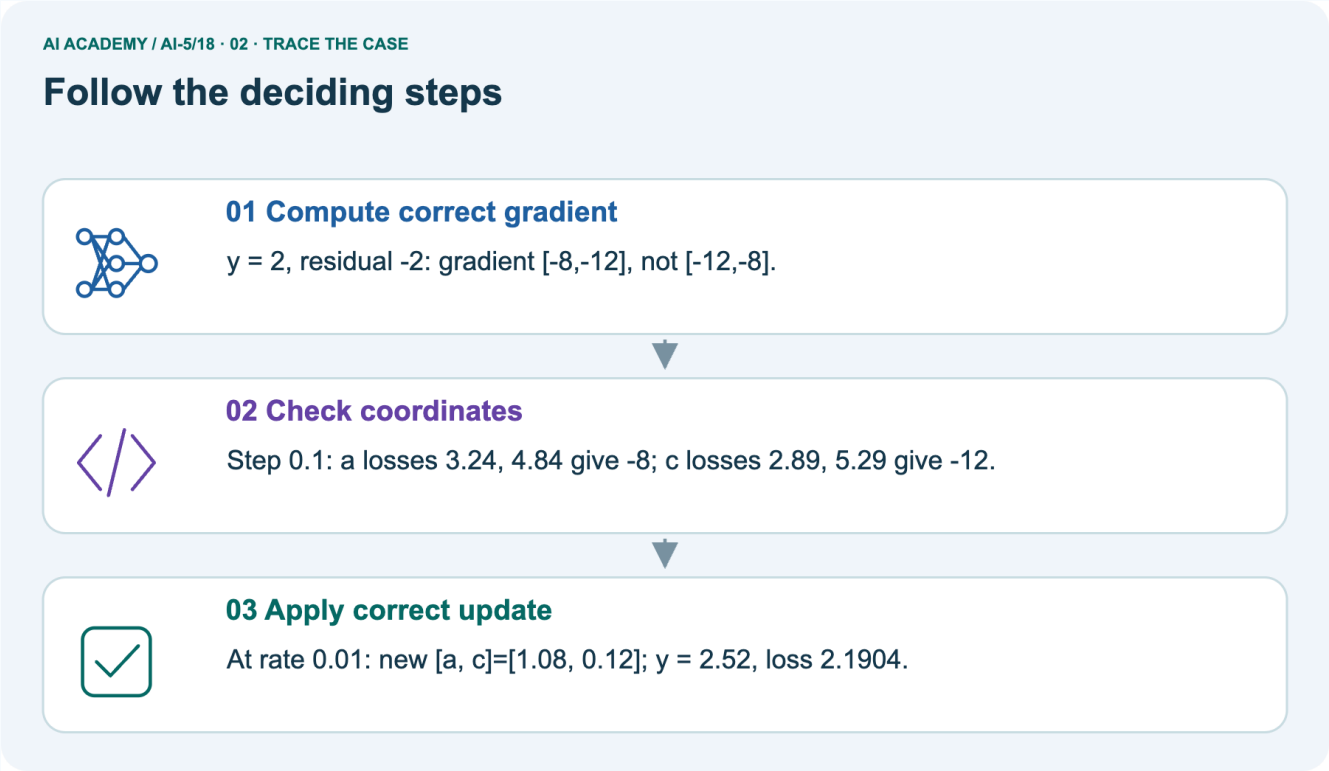
Follow the labelled picture with the matching explanation. Open the vector diagram

Catch a gradient -order bug before the optimiser changes parameters. Supplied facts: Toy y=2a+3c, target 4, start [a, c]=[1, 0]. Loss (y-4)². Program claims gradient [-12,-8] in order [a, c].

All **input**s and outcomes in this worked example are invented classroom data; no real model run is claimed.

Point to the given fact that matters most. Explain why it matters.

02. Follow the deciding steps



Follow the labelled picture with the matching explanation. Open the vector diagram

Compute correct gradient: y = 2, residual -2: gradient [-8,-12], not [-12,-8]. → Check coordinates: Step 0.1: a losses 3.24, 4.84 give -8; c losses 2.89, 5.29 give -12. → Apply correct update: At rate 0.01: new [a, c]=[1.08, 0.12]; y = 2.52, loss 2.1904.

Point to each numbered stage and explain which recorded **input** it uses.

03. Repair from the first disagreement

AI ACADEMY / AI-5/18 · 03 · INSPECT THE LESSON MODEL

Repair from the first disagreement

Trace stage for x [1 2]	Expected result	Submitted result
Input and dimensions	x length 2 W 3 by 2 b length 3	Dimensions match
Scores before ReLU	[1, 2, 0]	[1, 2, -1]
Hidden activations	[1, 2, 0]	[1, 2, 0]

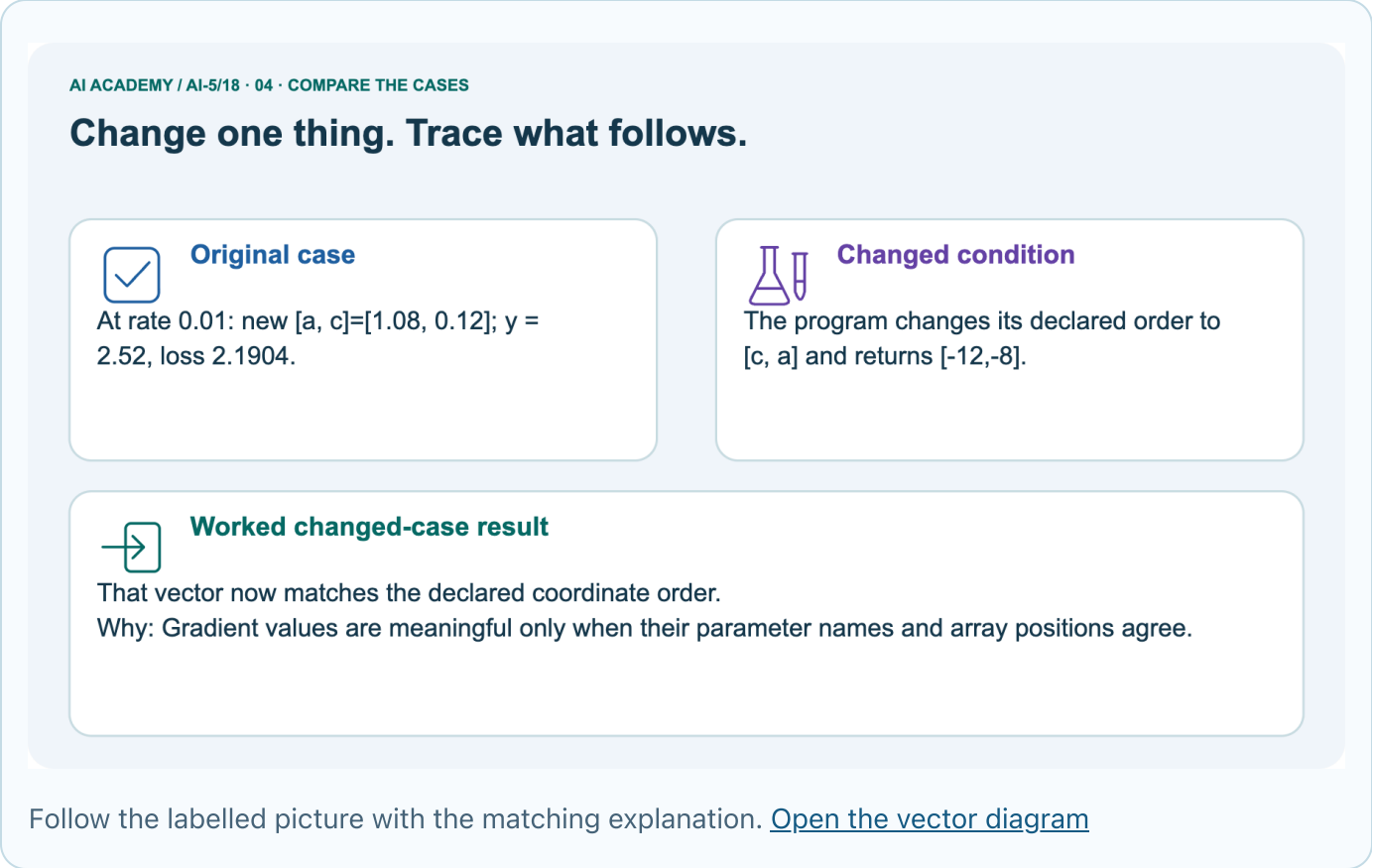
The submitted third score omits bias 1. ReLU hides the error for this input, so matching final activations are not enough. At x = [3, 1], the same omission changes the third activation from 3 to 2.

Follow the labelled picture with the matching explanation. Open the vector diagram

Repair from the first disagreement. **Input** and dimensions | x length 2 W 3 by 2 b length 3 | Dimensions match. Scores before ReLU | [1, 2, 0] | [1, 2, -1]. Hidden activations | [1, 2, 0] | [1, 2, 0]

Read the labels and values. Explain the relationship the model shows.

04. Change one thing. Trace what follows.



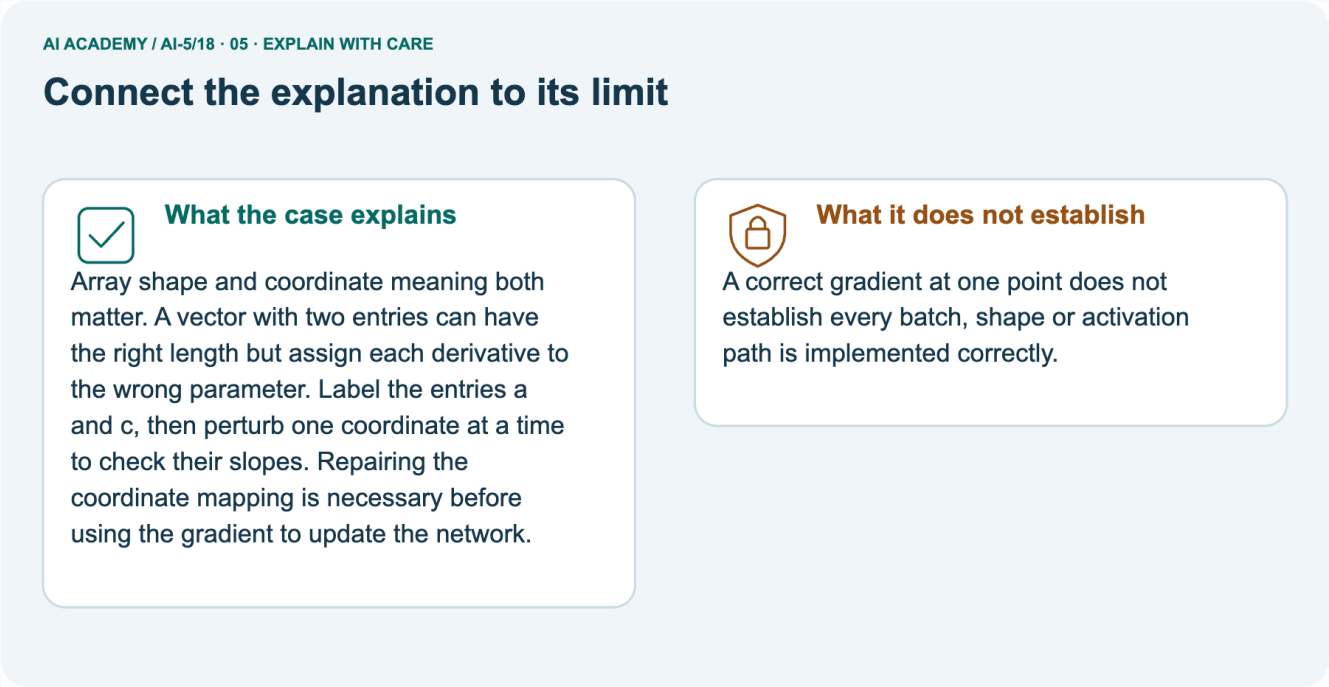
Follow the labelled picture with the matching explanation. Open the vector diagram

Original case: At rate 0.01: new [a, c]=[1.08, 0.12]; y = 2.52, loss 2.1904. Changed condition: The program changes its declared order to [c, a] and returns [-12,-8].

Expected result: That vector now matches the declared coordinate order. Reason: Gradient values are meaningful only when their parameter names and array positions agree.

Predict before checking the worked changed-case result. Name what changed and what the supplied facts allow you to conclude.

05. Connect the explanation to its limit



Follow the labelled picture with the matching explanation. Open the vector diagram

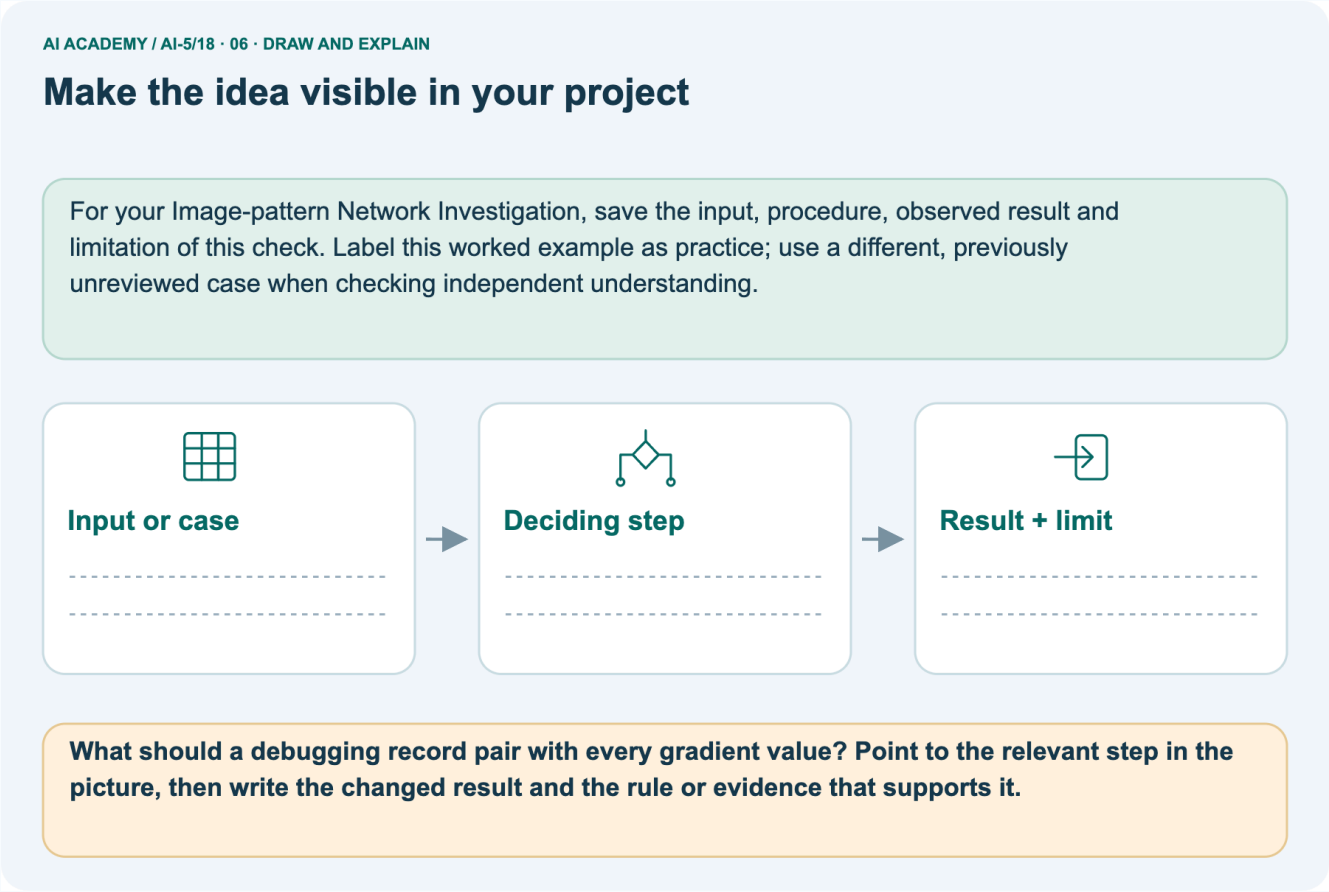
Array **shape** and coordinate meaning both matter. A vector with two entries can have the right length but assign each derivative to the wrong parameter.

Label the entries a and c, then perturb one coordinate at a time to check their slopes. Repairing the coordinate mapping is necessary before using the gradient to update the network.

Limit: A correct gradient at one point does not establish every batch, **shape** or activation path is implemented correctly.

Explain the case, then point to a limit on the claim.

06. Make the idea visible in your project



Follow the labelled picture with the matching explanation. Open the vector diagram

For your Image-pattern Network Investigation, save the **input**, procedure, observed result and limitation of this check.

Label this worked example as practice; use a different, previously unreviewed case when checking independent understanding. Draw your **input**, deciding step, result and limit.

Supported question: What should a debugging record pair with every gradient value? Point to the relevant step in the picture, then write the changed result and the rule or **evidence** that supports it.

Draw your project trace and justify the deciding step using **evidence**.

▼ Earlier wording, alternatives and picture transcripts

01. Debugging shapes and gradients: the case

02. Follow the deciding steps

03. Repair from the first disagreement

04. Change one thing. Trace what follows.

05. Connect the explanation to its limit

06. Make the idea visible in your project

▼ Optional interactive practice and completed work

Save or print your responses for your portfolio. Progress and drafts on the hosted website stay in the current browser. Use only fictional or teacher-approved data.